

<통신시스템 캡스톤 디자인1>

PC 서버 기반 YOLOv8n·YOLOv8s 실행
환경 구축 및 사람 탐지 비교 실험

5주차 보고서



-오석준-

체크리스트

DAY	주요 계획	목표	비고
3.31	Raspberry Pi OS 기반 환경 점검 SSH 원격 접속 구성 네트워크 및 IP 확인 Python/pip/venv 환경 구성, YOLO 설치 및 실행 가능성 검토	라즈베리파이 개발 환경 구축 완료 YOLO 실행 오류 (Illegal instruction) 확인	완료
4.7	PC 기반 FastAPI 서버 구축 기본 API(GET/POST) 작성 라즈베리파이↔서버 통신 확인	라즈베리파이와 PC 서버 간 통신 가능 상태 확보	완료
4.14	USB 웹캠(Logitech C920) 연결 카메라 인식 확인 OpenCV 기반 영상 캡처 및 이미지 저장 라즈베리파이에서 캡처한 이미지를 PC 서버로 전송 HTTP POST 방식 업로드 기능 구현 라이브 스트리밍 기능 구현	영상 입력 환경 구축 완료 이미지 전송 기능 구현	완료
5.8	PC 서버에서 YOLOv8n/v8s 실행 환경 구성 이미지 기반 객체 탐지 테스트	객체 탐지 기능 1차 구현	완료
5.12	등록 얼굴 데이터 수집, 얼굴 검출 및 특 정 추출 방식 설계, 등록 인물과 미등록 인물을 구분하기 위한 얼굴 인식 구조 설계, 결과 저장을 위한 DB 구조 설계	얼굴 인식 엔진 및 데 이터 저장 체계 확립	
5.19	Opencv 기반 얼굴 검출과 특징 비교 기 능 구현, 등록 인물/미등록 인물 판별 테 스트, yolo 사람 탐지 결과와 얼굴 인식 결과를 결합한 판별하는 로직을 구현	등록 인물과 외부인 구 분 기능 구현, 객체 탐 지와 얼굴 인식을 결합 한 판별 구조 확인	
5.26	이벤트 정의 및 알람 기능 구현 침입 발생 시 로그/알람 생성	이벤트 기반 시스템 완성	
6.2	전체 시스템 통합 테스트 오류 수정 및 성능 개선	1학기 1차 완성본 도출	

목 차

목 차

1. 프로젝트 개요 및 5주차 수행 내용
2. YOLO 기반 객체 탐지 개요
 - 2.1 YOLO 객체 탐지의 개념
 - 2.2 YOLOv8n/YOLOv8s 모델 선정 이유
 - 2.3 이미지·동영상·실시간 웹캠 객체 탐지 가능성
3. PC 서버 YOLO 실행 환경 구축
 - 3.1 작업 폴더 생성
 - 3.2 이미지 분류 및 저장 폴더 구성
 - 3.3 Python 가상환경 생성
 - 3.4 ultralytics 및 OpenCV 설치
4. 테스트 이미지 준비
 - 4.1 Raspberry Pi 촬영 이미지 활용
 - 4.2 이미지 분류 및 파일명 지정
5. 이미지 기반 객체 탐지 코드 작성
 - 5.1 코드 구성
 - 5.2 결과 저장 방식
 - 5.3 CSV 파일 생성
6. YOLOv8n/YOLOv8s 탐지 결과 분석

- 6.1 empty 이미지
- 6.2 person 이미지
- 6.3 multi_person 이미지
- 6.4 person_dark 이미지
- 6.5 오탐지 분석

7. YOLOv8n/YOLOv8s 비교 종합

8. 사람만 탐지하도록 코드 수정

9. 추가 확인: 동영상 및 실시간 웹캠 탐지

10. 결론 및 6주차 계획

1. 프로젝트 개요 및 5주차 수행내용

본 프로젝트는 Raspberry Pi와 USB 웹캠을 활용하여 실내 공간의 이미지를 수집하고, PC 서버에서 YOLO 기반 객체 탐지를 수행하여 사람의 출입 및 침입 여부를 판단하는 시스템을 구현하는 것을 목표로 한다.

최종적으로는 사람 객체 탐지 결과를 바탕으로 등록 인물과 미등록 인물을 구분하고, 침입 상황 발생 시 로그 저장 및 알림 기능으로 확장하고자 한다.

5주차에서는 PC 서버 환경에서 YOLOv8n과 YOLOv8s 모델의 실행 환경을 구축하고, Raspberry Pi에서 촬영하여 PC 서버로 전송한 이미지를 대상으로 사람 객체 탐지 테스트를 수행하였다.

또한 두 모델의 confidence, bounding box, 처리 시간 차이를 비교하여 실내 침입 감지 시스템에 적합한 모델을 검토하였다.

초기 실험에서는 YOLO 모델이 사람뿐만 아니라 냉장고, 의자, 책상 등 다양한 객체를 함께 탐지하는 것을 확인하였다.

그러나 본 프로젝트의 핵심 목적은 실내 침입 감지를 위한 사람 탐지이므로, 이후 `classes=[0]` 옵션을 적용하여 person 클래스만 탐지하도록 코드를 수정하였다.

추가적으로 동영상 및 실시간 웹캠 기반 객체 탐지를 확인하여, 정적 이미지 기반 사람 탐지 결과를 향후 실시간 영상 감시 시스템으로 확장할 수 있는 가능성을 검토하였다.

2. YOLO 기반 객체 탐지 개요

2.1 YOLO 객체 탐지의 개념

YOLO는 You Only Look Once의 약자로, 이미지나 영상 안에 존재하는 객체의 종류와 위치를 한 번에 탐지하는 딥러닝 기반 객체 탐지 모델이다.

YOLO는 객체의 위치를 사각형 영역인 bounding box로 표시하고, 해당 객체가 무엇인지 나타내는 class label과 탐지 신뢰도인 confidence 값을 함께 출력한다.

본 프로젝트에서는 실내 공간에서 사람이 존재하는지 확인하는 것이 중요하므로, YOLO를 활용하여 이미지 속 사람 객체를 탐지한다.

2.2 YOLOv8n/YOLOv8s 모델 선정 이유

YOLOv8 모델은 크기와 성능에 따라 여러 종류가 있으며, 대표적으로 YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, YOLOv8x 등이 있다.

이 중에서 본 실험에서는 YOLOv8n과 YOLOv8s를 사용하였다.

YOLOv8n은 nano 모델로, 모델 크기가 작고 처리 속도가 빠르다는 장점이 있다.

따라서 Raspberry Pi와 같은 소형 장치나 실시간 처리가 필요한 환경에서 사용 가능성을 검토하기에 적합하다.

반면 YOLOv8s는 small 모델로, YOLOv8n보다 모델 크기는 크지만 일반적으로 더 안정적인 탐지 성능을 기대할 수 있다.

단순히 정확도만 높은 모델보다 처리 속도와 탐지 성능의 균형이 중요하다.

이에 따라 가볍고 빠른 YOLOv8n과 상대적으로 탐지 성능이 높은 YOLOv8s를 비교하여, 실내 감시 환경에 더 적합한 모델을 검토하였다.

2.3 이미지·동영상·실시간 웹캠 객체 탐지 가능성

YOLO는 정적 이미지뿐만 아니라 동영상과 실시간 웹캠 영상에서도 객체 탐지가 가능하다.

정적 이미지 객체 탐지는 저장된 사진을 대상으로 탐지를 수행하는 방식이며, 실험 결과를 비교하고 분석하기에 적합하다.

반면 동영상 및 실시간 웹캠 객체 탐지는 연속적인 프레임에서 객체를 탐지하므로 실제 감시 시스템에 더 가까운 방식이다.

본 5주차 실험에서는 먼저 Raspberry Pi에서 촬영하여 PC 서버로 전송한 이미지를 대상으로 YOLOv8n과 YOLOv8s의 사람 탐지 결과를 비교하였다.

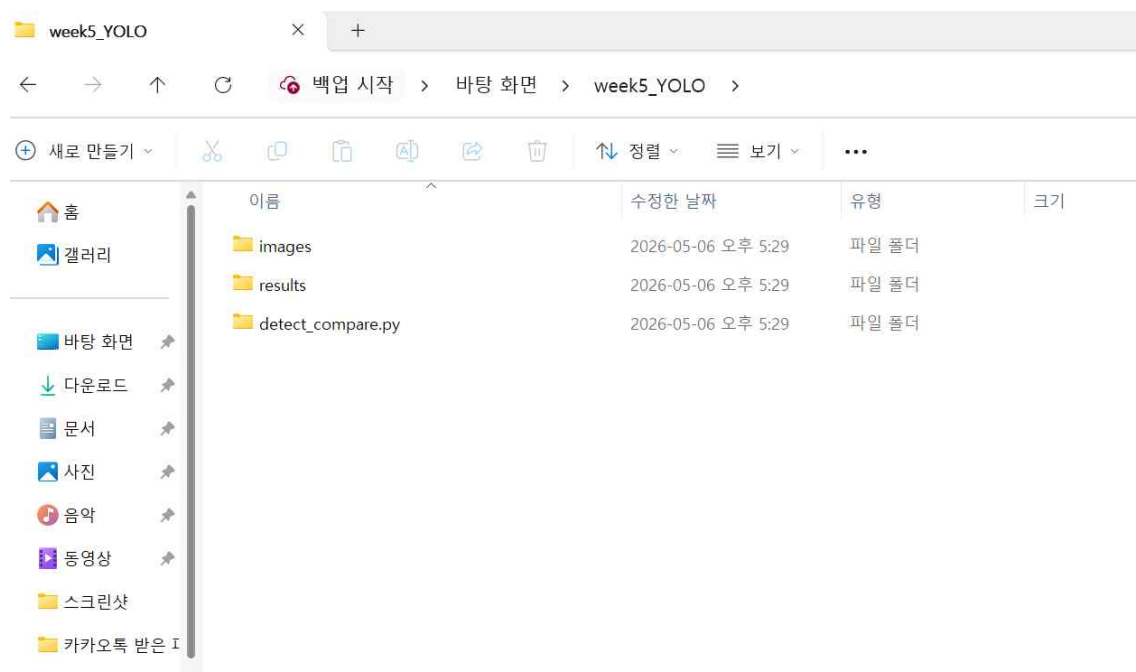
이후 추가적으로 동영상 및 실시간 웹캠 기반 객체 탐지를 확인함으로써, 정적 이미지 기반 실험 결과가 향후 실시간 침입 감시 시스템으로 확장될 수 있는지 검토하였다.

3. PC 서버 YOLO 실행 환경 구축

3-1 PC에 YOLO 객체 탐지 작업 폴더 생성

PC 서버에서 YOLO 객체 탐지를 수행하는 과정이므로, 먼저 PC에 별도의 작업 폴더를 생성하였다.

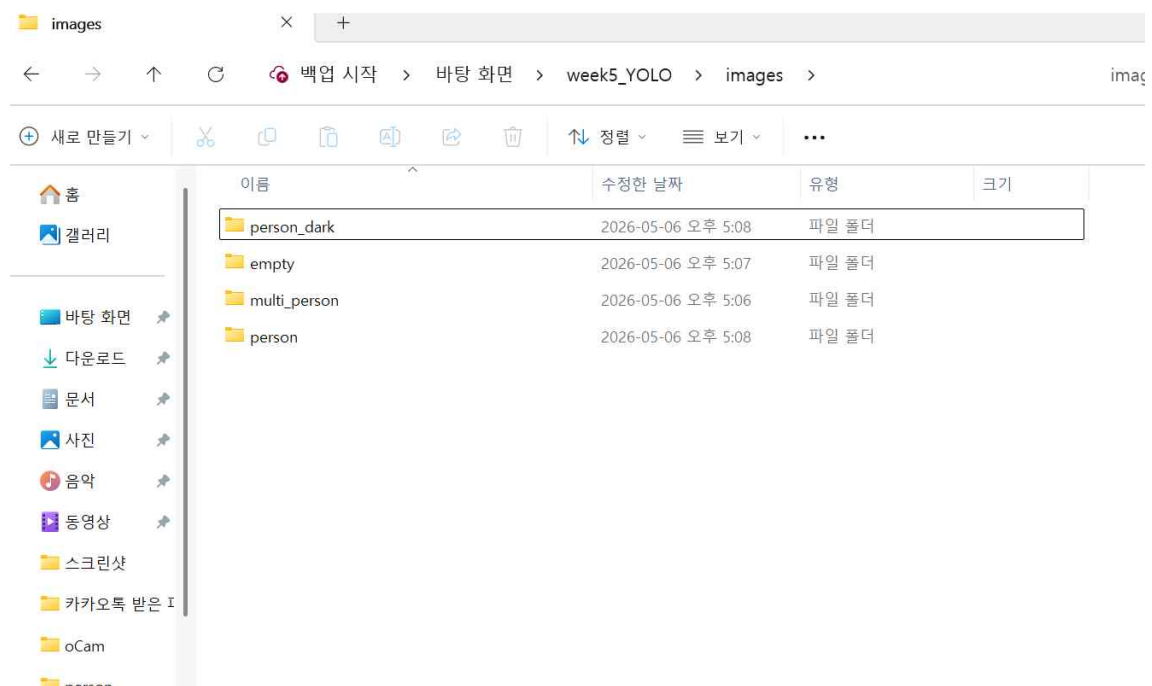
작업 폴더는 이미지 원본, 탐지 결과, 실행 코드가 혼동되지 않도록 구분하여 관리하기 위해 만들었다.



3-2 수집 이미지 분류 및 저장 폴더 구성

Raspberry Pi에서 촬영하여 PC 서버로 전송한 이미지를 상황별로 분류하였다.

이미지 분류는 YOLO 모델이 다양한 조건에서 사람 객체를 탐지할 수 있는지 확인하기 위한 것이다.



empty 사람이 없는 배경 이미지

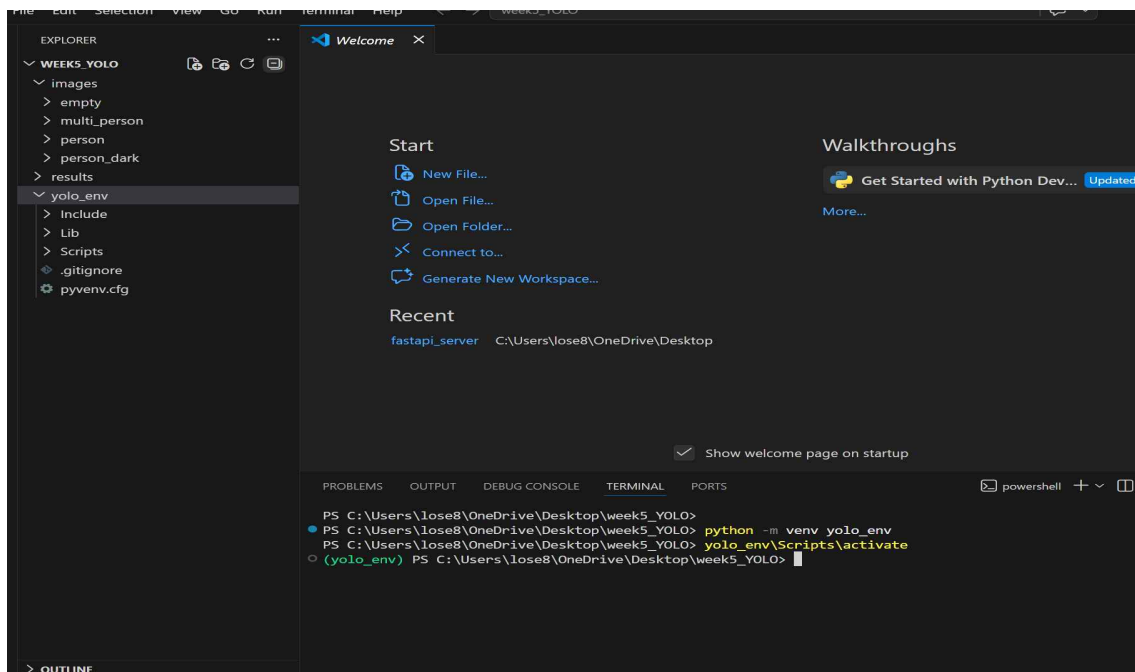
person 한 명의 사람이 포함된 이미지

multi_person 두 명 이상이 포함된 이미지

person_dark 어두운 조명 환경에서 촬영한 사람 이미지

3-3 Python 가상환경 생성 및 활성화

프로젝트 독립성 확보를 위해 Python 가상환경(yolo_env)을 생성하고 활성화하였다.

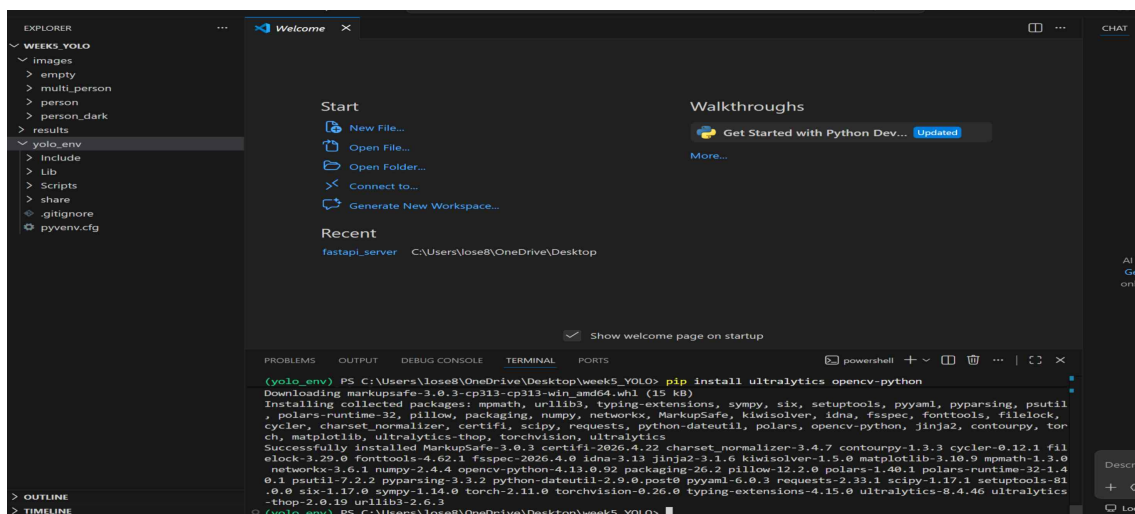


3-4 YOLO 실행을 위한 ultralytics, opencv-python 설치

YOLOv8 모델을 실행하기 위해 ultralytics 라이브러리를 설치하였다.

이미지 확인 및 향후 영상 프레임 처리와의 연계를 위해 OpenCV 라이브러리를 함께 설치하였다.

```
python -m pip install --upgrade pip  
pip install ultralytics opencv-python
```



The screenshot shows a Visual Studio Code interface with a terminal window open at the bottom. The terminal displays the command `pip install ultralytics opencv-python` and its output. The output indicates that several dependencies were collected and installed, including `markupsafe`, `pillow`, `numpy`, `torchvision`, `ultralytics`, and others. The installation was successful.

```
(yolo_env) PS C:\Users\iose8\OneDrive\Desktop\week5_YOLO> pip install ultralytics opencv-python  
Downloading markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)  
Installing collected packages: mpmath, urllib3, typing-extensions, sympy, six, setuptools, pyyaml, pyparsing, psutil, polars-runtime-32, pillow, packaging, numpy, networkx, MarkupSafe, kiwisolver, idna, fsspec, fonttools, filelock, cyclery, charset-normalizer, certifi, scipy, requests, python-dateutil, polars, opencv-python, Jinja2, contourpy, torch, matplotlib, ultralytics-thop, torchvision, ultralytics  
Successfully installed MarkupSafe-3.0.3 certifi-2026.4.22 charset-normalizer-3.4.7 contourpy-1.3.3 cyclery-0.12.1 filelock-3.29.0 fonttools-4.62.1 fsspec-2026.4.0 idna-3.13 Jinja2-3.1.6 kiwisolver-1.5.0 matplotlib-3.10.0 mpmath-1.3.0 networkx-3.6.1 numpy-2.4.4 opencv-python-4.13.0.92 packaging-26.2 pillow-12.2.0 polars-1.40.1 polars-runtime-32-1.40.1 psutil-7.2.2 pyparsing-3.3.2 python-dateutil-2.9.0.post0 pyyaml-6.0.3 requests-2.33.1 scipy-1.17.1 setuptools-81.0.0 six-1.17.0 sympy-1.14.0 torch-2.11.0 torchvision-0.26.0 typing-extensions-4.15.0 ultralytics-8.4.46 ultralytics-thop-2.0.19 urllib3-2.6.3
```

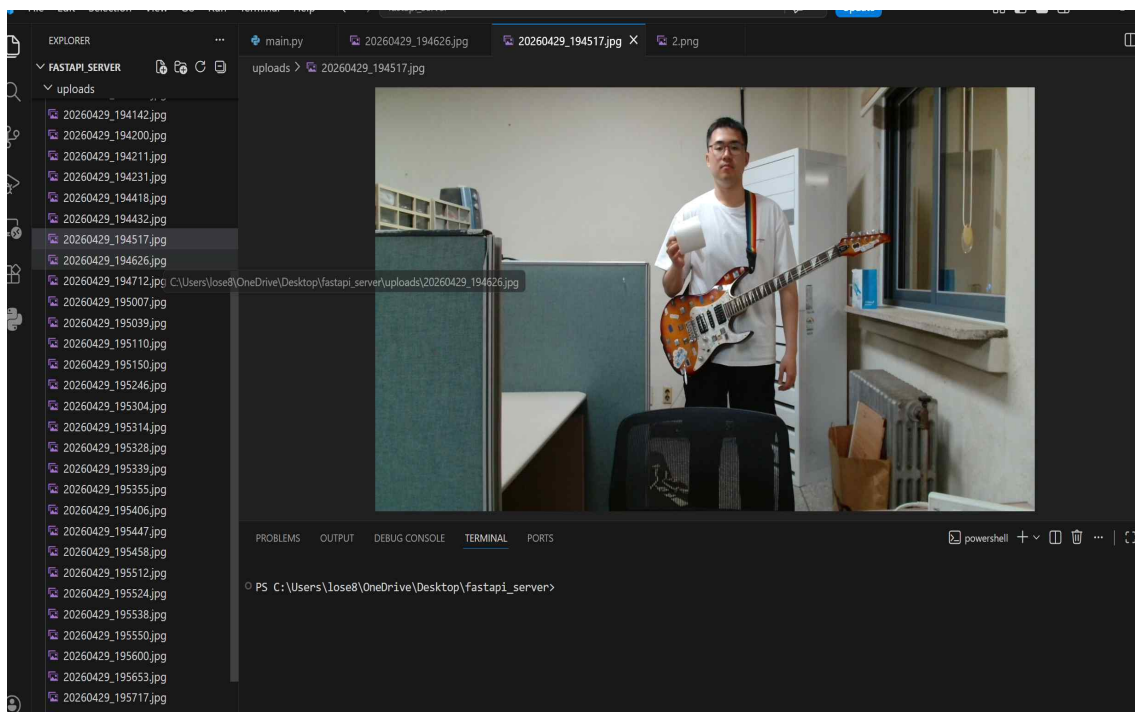
출처: [YOLO 설치](#)

4. 테스트 이미지 준비

4.1 Raspberry Pi 촬영 이미지 활용

4주차에서 구축한 Raspberry Pi 기반 이미지 캡처 및 FastAPI 서버 전송 구조를 활용하여 테스트 이미지를 확보하였다.

Raspberry Pi에 연결된 USB 웹캠으로 사람 포함 이미지와 배경 이미지를 촬영하고, PC FastAPI 서버의 uploads 폴더로 전송하였다.



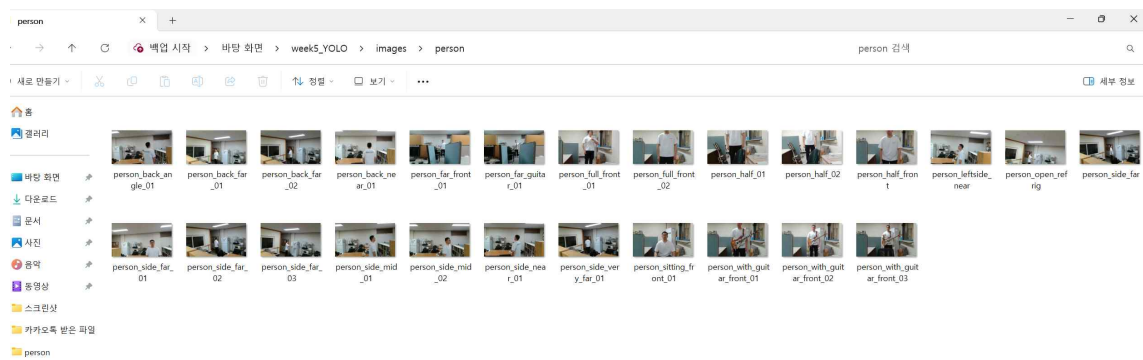
전송된 이미지는 5주차 YOLO 객체 탐지 테스트의 입력 이미지로 사용하였다.

4.2 이미지 분류 및 파일명 지정

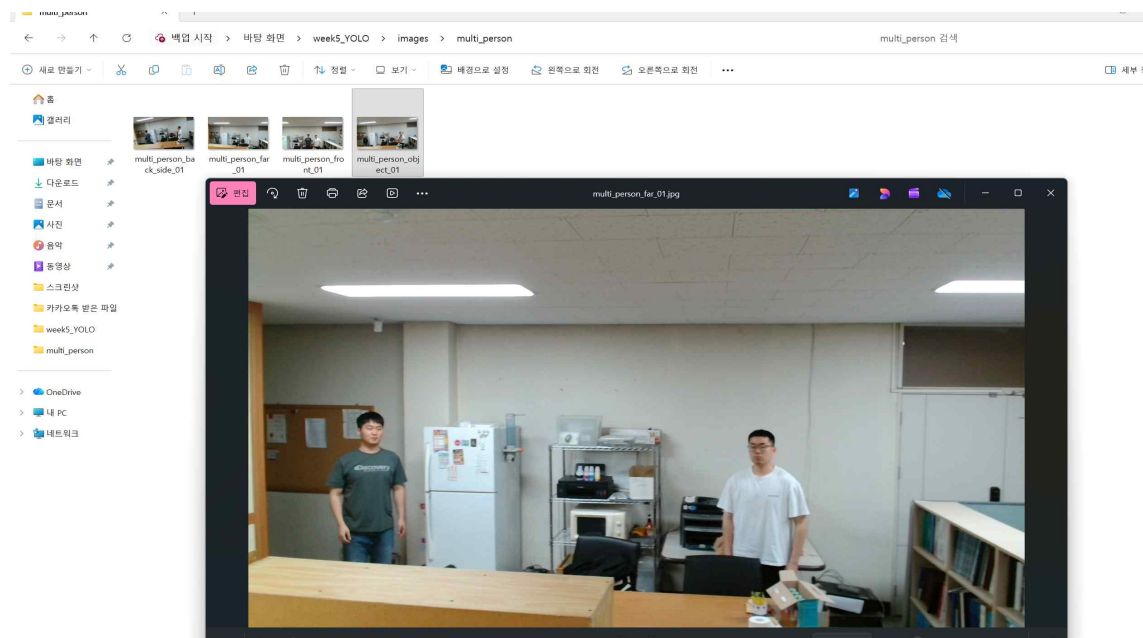
수집 이미지들을 YOLO 객체 탐지 테스트에 활용하기 위해 상황별로 분류하였다.

서버의 uploads 폴더에 저장된 원본 이미지는 실험 목적에 맞게 images 폴더로 복사하였으며, 사람의 위치, 거리, 자세, 소지 물체 여부를 구분할 수 있도록 파일명을 지정하였다.

예) person



예) multi_person



5. 이미지 기반 객체 탐지 코드 작성

5.1 코드 구성

PC 서버 환경에서 YOLOv8n과 YOLOv8s 모델을 이용하여 이미지 기반 객체 탐지를 수행하기 위해 detect_compare.py 코드를 작성하였다.

해당 코드는 images 폴더 하위에 분류해 둔 이미지들을 자동으로 불러와 YOLO 모델에 입력하고, 각 이미지에서 person 객체가 탐지되는지 확인하도록 구성하였다.

전체 코드

```
"""
본 코드에서는 ultralytics의 yolo 모듈을 사용하여 yolov8n과 yolov8s 모델을 불러왔다.
path는 이미지 및 결과 저장 경로 관리에 사용하였고,
time은 이미지별 처리 시간 측정
csv는 탐지 결과 저장을 위해 사용하였다.
"""

from ultralytics import YOLO
from pathlib import Path
import time
import csv

# 이미지 폴더
IMAGE_ROOT = Path("images")
# 결과 저장 폴더
RESULT_DIR = Path("results")
RESULT_DIR.mkdir(exist_ok=True) # results 폴더가 없으면 새로 만들고, 이미 있으면 그대로 사용
```

```

# 테스트할 모델
models ={
    "yolov8n": YOLO("yolov8n.pt"),
    "yolov8s": YOLO("yolov8s.pt"),
}
"""
이 부분은 images 폴더 안에 있는 모든 이미지 파일을 자동으로 찾아 목록에 넣는 부분이다.
코드는 각 폴더를 하나씩 확인한다.
예)empty 폴더에 있는 이미지는 category가 empty로 저장된다.
jpg, jpeg, png 파일만 탐지 대상으로 포함한다.
"""

# 이미지 목록 수집
image_files=[]
for category_dir in IMAGE_ROOT.iterdir():
    if category_dir.is_dir():
        category =category_dir.name
        for img_path in category_dir.glob("*"):
            if img_path.suffix.lower() in[".jpg", ".jpeg", ".png"]:
                image_files.append((img_path, category))

if not image_files:
    print("이미지가 없습니다. images 폴더 안의 하위 폴더를 확인하세요.")
    exit()

#결과 저장용 리스트 생성
summary_rows=[]
#나중에 이 리스트 안의 내용을 CSV 파일로 저장한다.
#YOLOv8n과 YOLOv8s를 차례대로 실행하는 부분이다.

for model_name, model in models.items():
    print(f"\n===== {model_name}테스트 시작 =====")

```

```

"""
이 부분이 실제 YOLO 탐지를 수행하는 핵심 부분
start_time = time.time()은 탐지 시작 시간을 기록
model(str(img_path), save=True, ...)는 해당 이미지를 YOLO 모델에 입력하여 객
체 탐지를 수행
save=True는 탐지 결과 이미지를 저장
project="results"는 결과를 results 폴더에 저장
name=model_name은 모델별로 결과 폴더를 나누기 위한 설정
"""

    for img_path, category in image_files:
        start_time =time.time()
        # YOLO 객체 탐지 실행
        results =model(
            str(img_path),
            save=True,
            project="results",
            name=model_name,
            exist_ok=True #기존 폴더를 그대로 사용해서 결과 저장
        )
"""
end_time = time.time()은 탐지 종료 시간을 기록
processing_time은 탐지에 걸린 시간을 초 단위로 계산한 값
"""

        end_time =time.time()
        processing_time =round(end_time -start_time, 3)

"""

이 부분은 각 이미지별 탐지 결과를 저장하기 위한 초기값
person_detected는 사람이 탐지되었는지를 나타낸다.
처음에는 False로 시작하고, person 객체가 발견되면 True로 바뀐다.
max_conf는 탐지된 사람 중 가장 높은 confidence 값을 저장한다.
bbox_list는 탐지된 사람의 bounding box 좌표를 저장
"""

```

```

person_detected = False
max_conf = None
bbox_list = []

#YOLO는 이미지에서 탐지한 객체 정보를 result.bboxes 안에 저장한다.
#예를 들어 YOLO가 사람을 찾으면 label이 person으로 나온다.

for result in results:
    for box in result.bboxes:
        cls_id = int(box.cls[0]) #box.cls: 객체 클래스 번호
        conf = float(box.conf[0]) # box.conf: 탐지 신뢰도
        label = model.names[cls_id] #model.names[cls_id]: 클래스 이름
        bbox = [round(x, 2) for x in box.xyxy[0].tolist()]
        #box.xyxy: bounding box 좌표
        """
bbox는 사람 위치를 나타내는 좌표이다.
[x1, y1, x2, y2]
x1, y1: 박스의 왼쪽 위 좌표
x2, y2: 박스의 오른쪽 아래 좌표
"""

"""
YOLO는 사람뿐만 아니라 의자, 책상, 가방, 컵, 기타 등 다양한 객체를 탐지할 수 있다.
하지만 이번 실험의 목표는 사람 탐지이므로, label이 person인 객체만 따로 확인한다.
person_detected를 True로 변경
사람 위치 좌표를 bbox_list에 저장
여러 사람이 탐지된 경우 가장 높은 confidence 값을 max_conf에 저장
"""

```

```

        if label == "person":
            person_detected = True
            bbox_list.append(bbox)
            if max_conf is None or conf > max_conf:
                max_conf = conf
    """
이 부분은 탐지 결과를 사람이 이해하기 쉽게 해석하는 부분
이미지 상황과 YOLO 결과를 비교하여 네 가지로 분류"""

    if category == "empty" and not person_detected:
        result_text = "오탐 없음"
    elif category == "empty" and person_detected:
        result_text = "오탐 발생"
    elif category != "empty" and person_detected:
        result_text = "정상 탐지"
    else:
        result_text = "미탐지"

#터미널에 탐지 결과를 출력하는 부분
#위에 설명한 저장되는 정보들
print(f"이미지: {img_path.name}")
print(f"분류: {category}")
print(f"모델: {model_name}")
print(f"person 탐지 여부: {'0' if person_detected else 'X'}")
print(f"confidence: {round(max_conf, 3) if max_conf is not None
else '-'}")
print(f"bbox: {bbox_list if bbox_list else '-'}")
print(f"처리 시간: {processing_time}초")
print(f"결과: {result_text}")
print("-"*40)
#출력된 탐지 결과를 CSV 파일로 저장하기 위해 리스트에 추가
#나중에 Excel에서 열 수 있도록 각 결과를 한 줄씩 정리
summary_rows.append({
    "image": img_path.name,
    "category": category,
    "model": model_name,
    "person_detected": "0" if person_detected else "X",
    "confidence": round(max_conf, 3) if max_conf is not None else
    "-",

```

```

        "bbox": bbox_list if bbox_list else "-",
        "processing_time_sec": processing_time,
        "result": result_text,
    })
# 전체 탐지 결과를 CSV 파일로 저장
csv_path =RESULT_DIR /"yolo_compare_summary.csv"

with open(csv_path, "w", newline="", encoding="utf-8-sig") as f:
    fieldnames =[
        "image",
        "category",
        "model",
        "person_detected",
        "confidence",
        "bbox",
        "processing_time_sec",
        "result",
    ]

    writer =csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(summary_rows)
#코드 실행이 완료되면 전체 테스트 완료 메시지와 함께 CSV 저장 위치 및 결과 이미
지 저장 폴더를 출력
print("\n전체 테스트 완료")
print(f"결과 CSV 저장 위치: {csv_path}")
print("결과 이미지는 results/yolov8n, results/yolov8s 폴더에서 확인하세요.")

```

5.2 결과 저장 방식

이번 코드는 상황별로 분류한 이미지를 자동으로 불러와 YOLOv8n과 YOLOv8s 모델에 각각 입력하는 방식으로 구성했습니다.

초기 실험에서는 YOLO가 탐지한 여러 객체 중 person 객체만 선별하여 분석하였다.

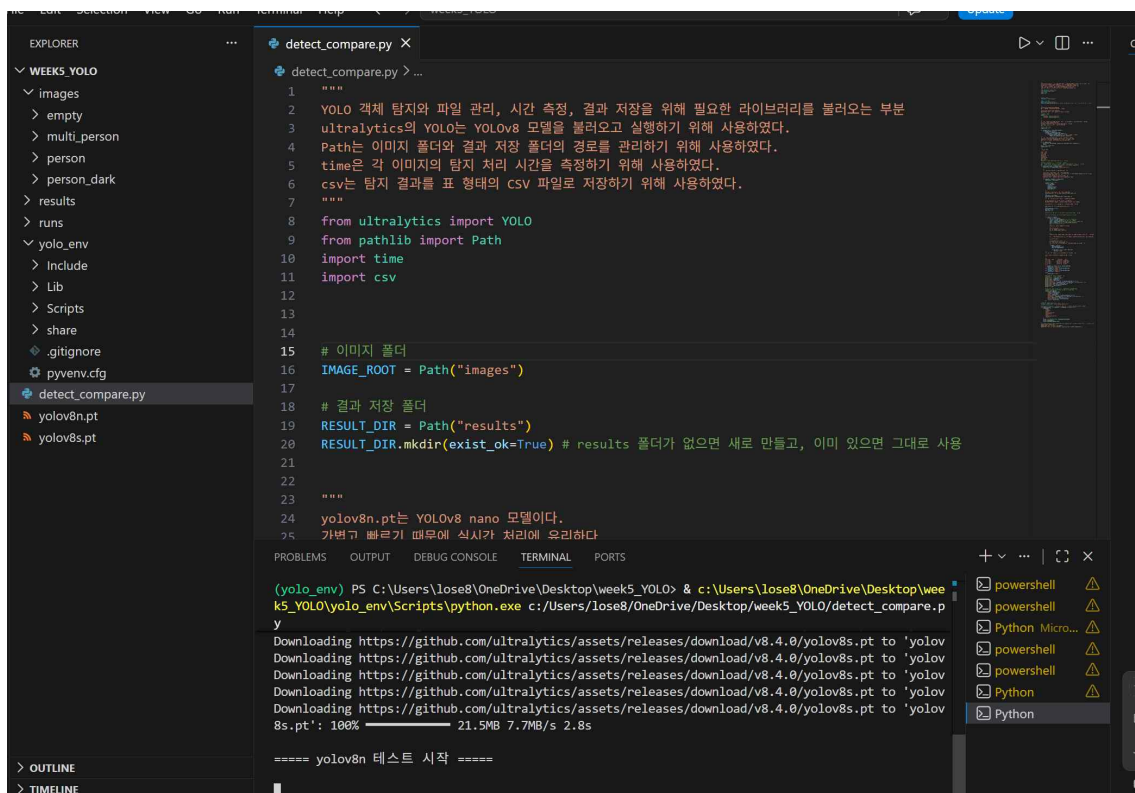
이후 본 프로젝트 목적에 맞게 classes=[0] 옵션을 추가하여 탐지 단계에서부터

사람 객체만 검출되도록 코드를 수정하였다.

결과 이미지는 모델별 폴더에 저장하고, 전체 결과는 CSV 파일로 저장하여 비교할 수 있도록 했습니다.

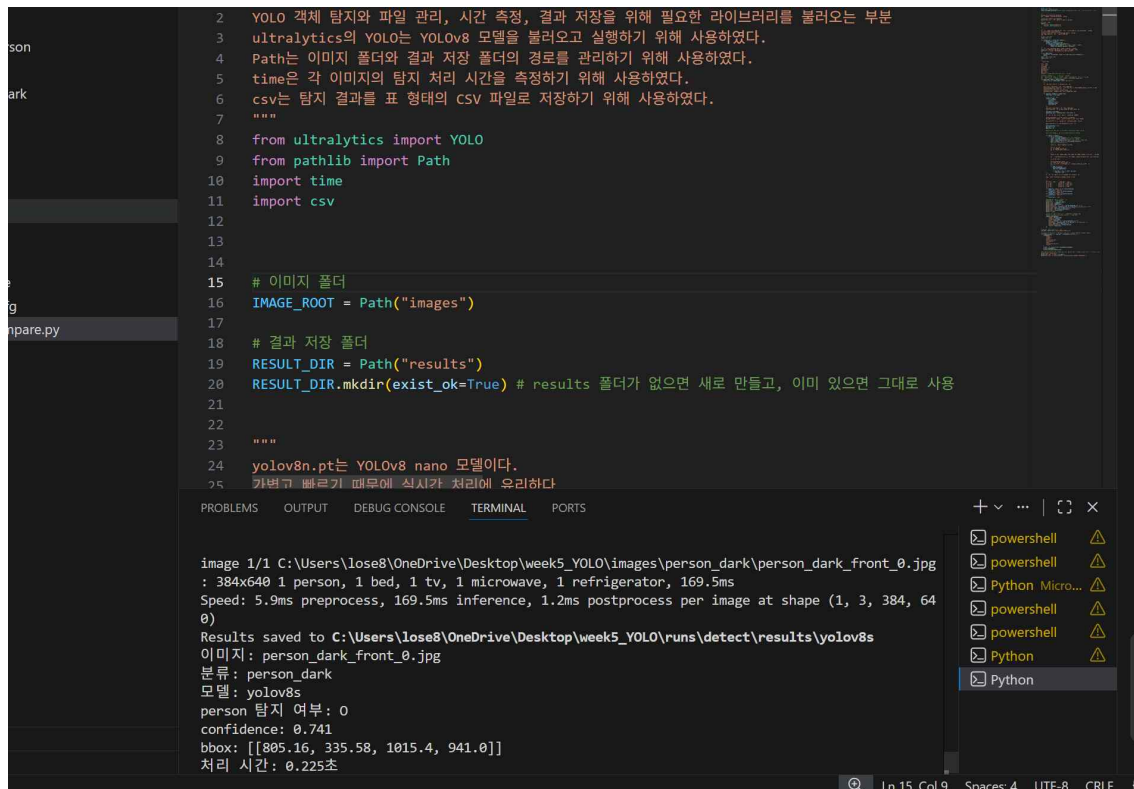
실행 화면

1) 실행 초기



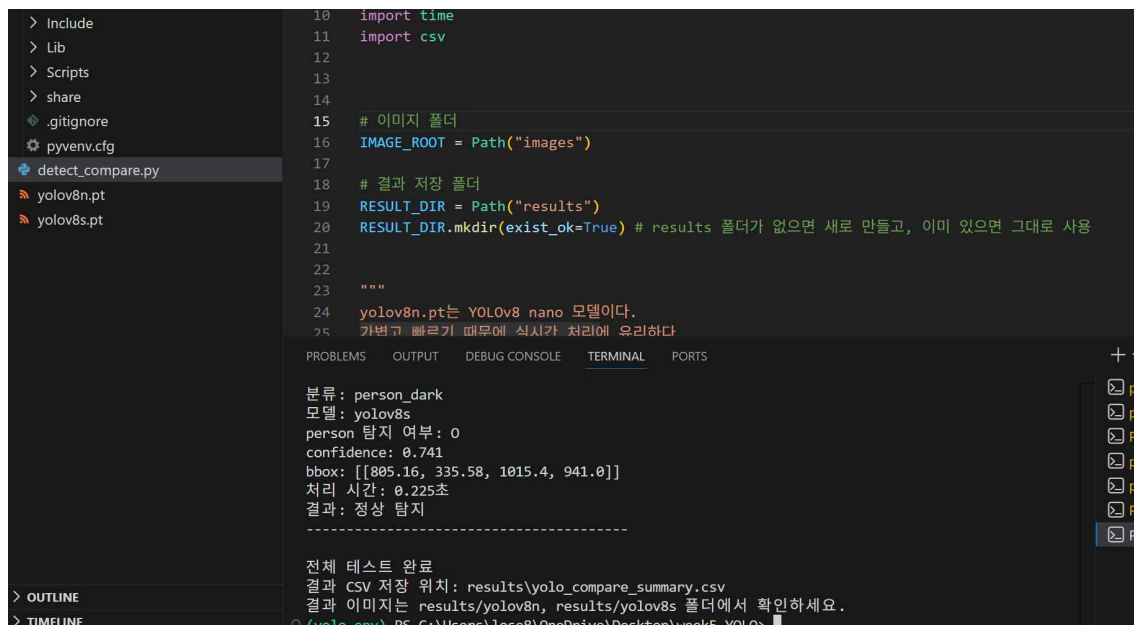
```
1 """
2 YOLO 객체 탐지와 파일 관리, 시간 측정, 결과 저장을 위해 필요한 라이브러리를 불러오는 부분
3 ultralytics의 YOLO는 YOLOv8 모델을 불러오고 실행하기 위해 사용하였다.
4 Path는 이미지 폴더와 결과 저장 폴더의 경로를 관리하기 위해 사용하였다.
5 time은 각 이미지의 탐지 처리 시간을 측정하기 위해 사용하였다.
6 csv는 탐지 결과를 표 형태의 csv 파일로 저장하기 위해 사용하였다.
7 """
8 from ultralytics import YOLO
9 from pathlib import Path
10 import time
11 import csv
12
13
14
15 # 이미지 폴더
16 IMAGE_ROOT = Path("images")
17
18 # 결과 저장 폴더
19 RESULT_DIR = Path("results")
20 RESULT_DIR.mkdir(exist_ok=True) # results 폴더가 없으면 새로 만들고, 이미 있으면 그대로 사용
21
22
23 """
24 yolov8n.pt는 YOLOv8 nano 모델이다.
25 가볍고 빠르기 때문에 실시간 처리에 유리하다
26
27 (yolo_env) PS C:\Users\lose8\OneDrive\Desktop\week5_YOLO> & c:\Users\lose8\OneDrive\Desktop\week5_YOLO\yolo_env\Scripts\python.exe c:/Users/lose8/OneDrive/Desktop/week5_YOLO/detect_compare.py
28
29 Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8s.pt to 'yolov
30 Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8s.pt to 'yolov
31 Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8s.pt to 'yolov
32 Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8s.pt to 'yolov
33 Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8s.pt to 'yolov
34 8s.pt': 100% 21.5MB 7.7MB/s 2.8s
35
36 ===== yolov8n 테스트 시작 =====
```

2) 이미지 하나 YOLO 모델 처리 결과



```
2 YOLO 객체 탐지와 파일 관리, 시간 측정, 결과 저장을 위해 필요한 라이브러리를 불러오는 부분
3 ultralytics의 YOLO는 YOLOv8 모델을 불러오고 실행하기 위해 사용하였다.
4 Path는 이미지 폴더와 결과 저장 폴더의 경로를 관리하기 위해 사용하였다.
5 time은 각 이미지의 탐지 처리 시간을 측정하기 위해 사용하였다.
6 csv는 탐지 결과를 표 형태의 csv 파일로 저장하기 위해 사용하였다.
7
8 from ultralytics import YOLO
9 from pathlib import Path
10 import time
11 import csv
12
13
14
15 # 이미지 폴더
16 IMAGE_ROOT = Path("images")
17
18 # 결과 저장 폴더
19 RESULT_DIR = Path("results")
20 RESULT_DIR.mkdir(exist_ok=True) # results 폴더가 없으면 새로 만들고, 이미 있으면 그대로 사용
21
22
23
24 """
25 yolov8n.pt는 YOLOv8 nano 모델이다.
26 가볍고 빠르기 때문에 실시간 처리에 유리하다
27 """
28
29 image 1/1 C:\Users\lose8\OneDrive\Desktop\week5_YOLO\images\person_dark\person_dark_front_0.jpg
30 : 384x640 1 person, 1 bed, 1 tv, 1 microwave, 1 refrigerator, 169.5ms
31 Speed: 5.9ms preprocess, 169.5ms inference, 1.2ms postprocess per image at shape (1, 3, 384, 640)
32 Results saved to C:\Users\lose8\OneDrive\Desktop\week5_YOLO\runs\detect\results\yolov8s
33 이미지: person_dark_front_0.jpg
34 분류: person_dark
35 모델: yolov8s
36 person 탐지 여부: 0
37 confidence: 0.741
38 bbox: [[805.16, 335.58, 1015.4, 941.0]]
39 처리 시간: 0.225초
```

3) 최종 결과



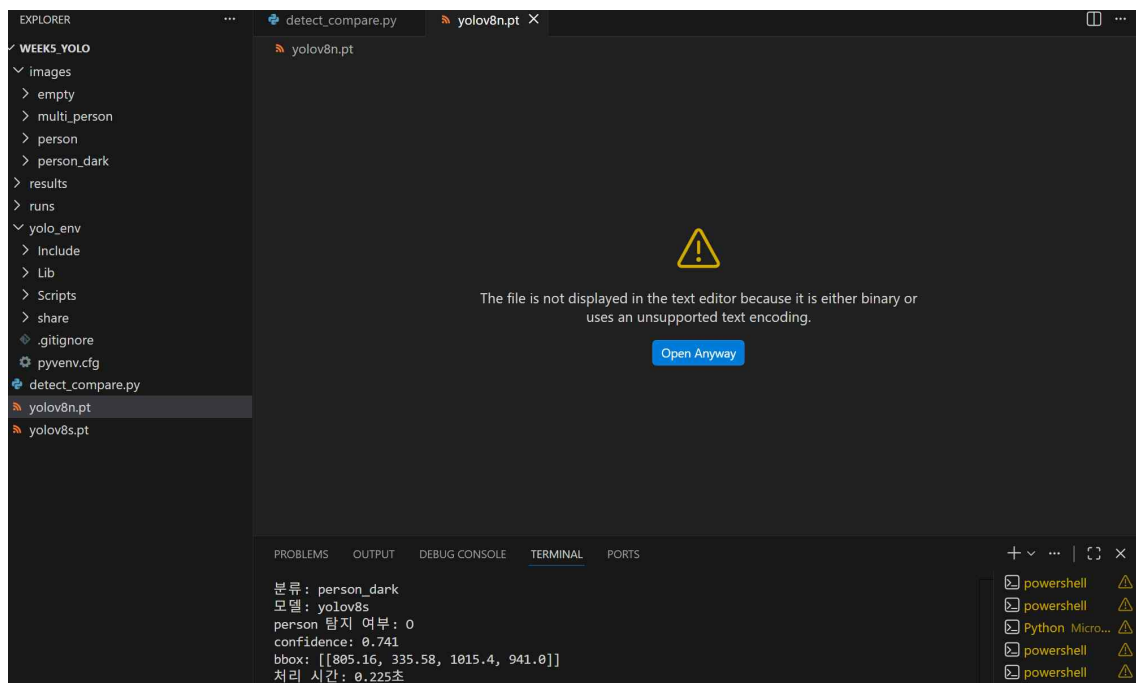
```
10 import time
11 import csv
12
13
14
15 # 이미지 폴더
16 IMAGE_ROOT = Path("images")
17
18 # 결과 저장 폴더
19 RESULT_DIR = Path("results")
20 RESULT_DIR.mkdir(exist_ok=True) # results 폴더가 없으면 새로 만들고, 이미 있으면 그대로 사용
21
22
23
24 """
25 yolov8n.pt는 YOLOv8 nano 모델이다.
26 가볍고 빠르기 때문에 실시간 처리에 유리하다
27 """
28
29 분류: person_dark
30 모델: yolov8s
31 person 탐지 여부: 0
32 confidence: 0.741
33 bbox: [[805.16, 335.58, 1015.4, 941.0]]
34 처리 시간: 0.225초
35 결과: 정상 탐지
36
37 -----
38
39 전체 테스트 완료
40 결과 CSV 저장 위치: results\yolo_compare_summary.csv
41 결과 이미지는 results\yolov8n, results\yolov8s 폴더에서 확인하세요.
42 (yolo env) PS C:\Users\lose8\OneDrive\Desktop\week5_YOLO>
```

참고)

YOLOv8n.pt와 YOLOv8s.pt는 YOLO 모델의 학습된 가중치 파일이다.

일반 텍스트 코드 파일이 아니라 모델 파라미터가 저장된 binary 파일이므로 VS Code에서 직접 내용을 확인할 수 없다.

본 실험에서는 두 가중치 파일이 정상적으로 로드되었고, 이미지 기반 사람 탐지에 사용되었다.



출처: [YOLO 객체 탐지 설명 코드와 다양한 방법들](#)

5.3 CSV 파일 생성

CSV 파일은 각 이미지별 탐지 결과를 정량적으로 비교하기 위해 생성하였다.

이를 통해 단순히 결과 이미지를 육안으로 확인하는 것에서 그치지 않고, 모델별 confidence와 처리 시간을 수치로 비교할 수 있었다.

	A	B	C	D	E	F	G	H	I
1	image	category	model	person_de	confidence	bbox	processing	result	
2	empty_mc	empty	yolov8n	X	-	-	0.824	오탐 없음	
3	empty_mc	empty	yolov8n	X	-	-	0.558	오탐 없음	
4	empty_ro	empty	yolov8n	X	-	-	0.304	오탐 없음	
5	empty_ro	empty	yolov8n	X	-	-	0.15	오탐 없음	
6	sample_yc	empty	yolov8n	O	0.722	[[54.11, 73	0.149	오탐 발생	
7	multi_pers	multi_pers	yolov8n	O	0.904	[[358.48, 4	0.124	정상 탐지	
8	multi_pers	multi_pers	yolov8n	O	0.913	[[194.19, 5	0.146	정상 탐지	
9	multi_pers	multi_pers	yolov8n	O	0.912	[[984.62, 6	0.263	정상 탐지	
10	multi_pers	multi_pers	yolov8n	O	0.902	[[153.68, 5	0.135	정상 탐지	
11	person_ba	person	yolov8n	O	0.929	[[1018.08,	0.131	정상 탐지	
12	person_ba	person	yolov8n	O	0.89	[[887.2, 60	0.164	정상 탐지	
13	person_ba	person	yolov8n	O	0.858	[[900.94, 5	0.27	정상 탐지	
14	person_ba	person	yolov8n	O	0.918	[[870.84, 3	0.173	정상 탐지	
15	person_fa	person	yolov8n	O	0.887	[[979.59, 3	0.277	정상 탐지	
16	person_fa	person	yolov8n	O	0.84	[[942.68, 3	0.189	정상 탐지	
17	person_fu	person	yolov8n	O	0.941	[[756.59, 1	0.19	정상 탐지	
18	person_fu	person	yolov8n	O	0.953	[[754.79, 1	0.181	정상 탐지	
19	person_ha	person	yolov8n	O	0.899	[[537.68, 3	0.298	정상 탐지	
20	person_ha	person	yolov8n	O	0.927	[[633.72, 4	0.256	정상 탐지	
21	person_ha	person	yolov8n	O	0.949	[[697.5, 1.9	0.288	정상 탐지	
22	person_le	person	yolov8n	O	0.904	[[834.42, 4	0.169	정상 탐지	
23	person_o	person	yolov8n	O	0.895	[[725.85, 7	0.137	정상 탐지	
24	person_si	person	yolov8n	O	0.846	[[964.05, 4	0.183	정상 탐지	
25	person_si	person	yolov8n	O	0.935	[[1086.32,	0.131	정상 탐지	
26	person_si	person	yolov8n	O	0.864	[[903.08, 4	0.166	정상 탐지	
27	person_si	person	yolov8n	O	0.861	[[781.2, 56	0.154	정상 탐지	
28	person_si	person	yolov8n	O	0.897	[[1088.37,	0.13	정상 탐지	
29	person_si	person	yolov8n	O	0.904	[[1055.47,	0.13	정상 탐지	

yolov8n

7	multi_pers	multi_pers	yolov8n	O	0.904	[[358.48, 4	0.124	정상 탐지
8	multi_pers	multi_pers	yolov8n	O	0.913	[[194.19, 5	0.146	정상 탐지

yolov8s

43	multi_pers	multi_pers	yolov8s	O	0.862	[[356.24, 4	0.209	정상 탐지
44	multi_pers	multi_pers	yolov8s	O	0.933	[[194.12, 5	0.349	정상 탐지

첫 번째 이미지에서 YOLOv8n이 0.124초, YOLOv8s가 0.209초로 YOLOv8n이 더 빠르게 처리되었으며, 두 번째 이미지에서도 YOLOv8n은 0.146초, YOLOv8s는 0.349초로 YOLOv8n의 처리속도가 더 우수하였다.

confidence 값은 이미지에 따라 차이가 있었다.

첫 번째 이미지에서는 YOLOv8n이 0.904로 YOLOv8s의 0.862보다 높았고, 두 번째 이미지에서는 YOLOv8s가 0.933으로 YOLOv8n의 0.913보다 높았다.

따라서 탐지 신뢰도는 두 모델 모두 양호하였으나, 실시간 처리 관점에서는 YOLOv8n이 더 적합한 결과를 보였다. 하지만 다른 이외의 이미지에서는 yolov8s가 신뢰도가 더 높게 나오는 경향이 많았다.

YOLOv8n과 YOLOv8s 모두 사람을 정상 탐지하였으며, YOLOv8n은 처리속도 측면에서 유리하고 YOLOv8s는 일부 이미지에서 신뢰도가 더 높게 나타나는 특징을 보였다.

6. YOLOv8n/YOLOv8s 탐지 결과 분석

본 실험에서는 탐지 결과를 ① 사람 탐지 여부, ② confidence 값, ③ bounding box 위치, ④ 처리 시간, ⑤ 오탐지 여부를 기준으로 분석하였다.

특히 본 프로젝트의 목적이 실내 침입 감지이므로, 최종 판단 기준은 주변 사물이 아니라 person 객체의 탐지 여부로 설정하였다.

1) 사람이 없는 이미지

사람이 없는 이미지에서는 person 객체가 탐지되지 않았다.
다만 주변 사물이 tv, keyboard, laptop, oven 등으로 탐지되었다.

YOLOv8n 결과에서는 모니터가 tv 0.40, 키보드가 keyboard 0.85 정도로 탐지되었다.

YOLOv8s 결과에서는 유사한 이미지에서 tv 0.51, tv 0.45, keyboard 0.92로 탐지되었다.

즉 사람이 없는 경우에는 사람 오탐지는 거의 없었지만, 주변 사물은 다양한 클래스로 탐지되었다.

yolov8n



yolov8s

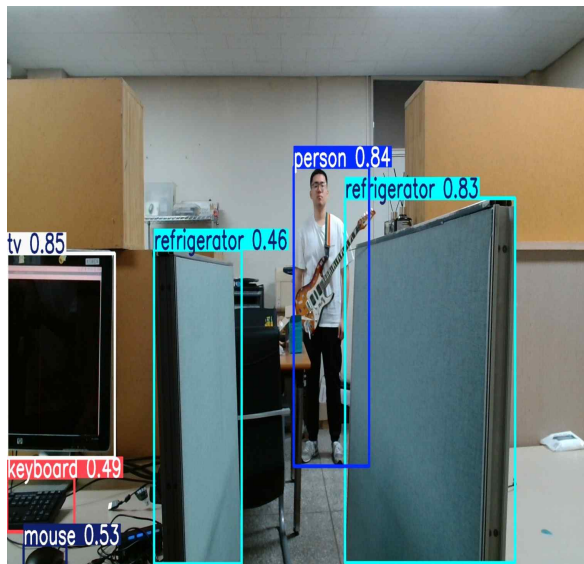


2) 사람이 1명 있는 이미지

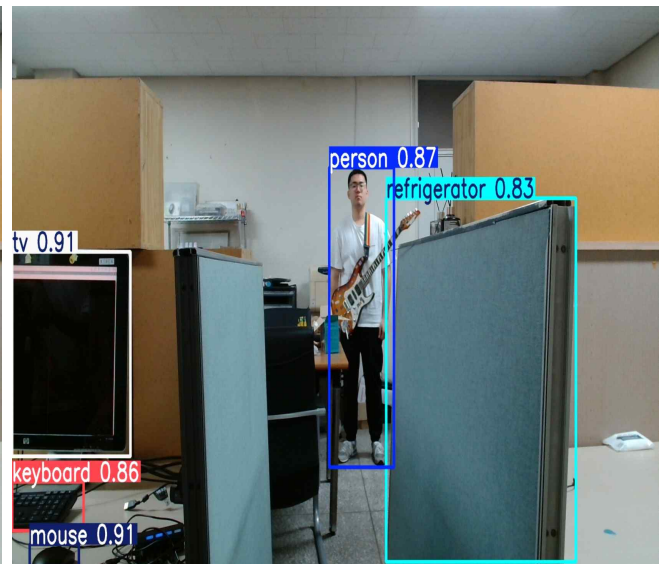
. YOLOv8n은 정면 이미지에서 person 0.91, person 0.87, person 0.90을 보였고, YOLOv8s는 person 0.93, person 0.94, person 0.95 수준을 보였다. 측면 및 후면 자세에서도 두 모델 모두 person 0.86~0.93 범위로 탐지하였다.

--> 두 모델 다 얼굴이 보이지 않더라도 사람의 신체 형태를 기준으로 탐지가 가능함을 확인하였다.

yolov8n



yolov8s



3) 사람이 2명 있는 이미지

두 명이 함께 있는 이미지에서도 두 모델 모두 사람 객체를 개별적으로 탐지하였다. YOLOv8n은 두 사람을 각각 person 0.91, person 0.87로 탐지하였고, YOLOv8s는 유사한 이미지에서 person 0.93, person 0.88로 탐지하였다. 이를 통해 단일 인물뿐 아니라 다중 인물이 포함된 상황에서도 사람 탐지가 가능함을 확인하였다.

yolov8n



yolov8s



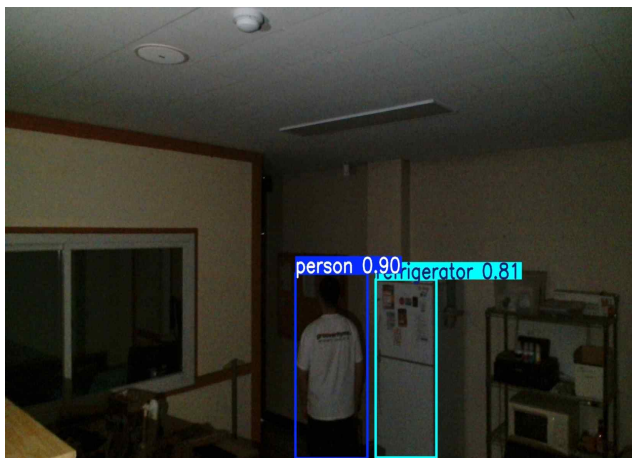
4) 어두운 환경 이미지

어두운 환경 이미지에서도 두 모델 모두 사람 객체를 탐지하였다.

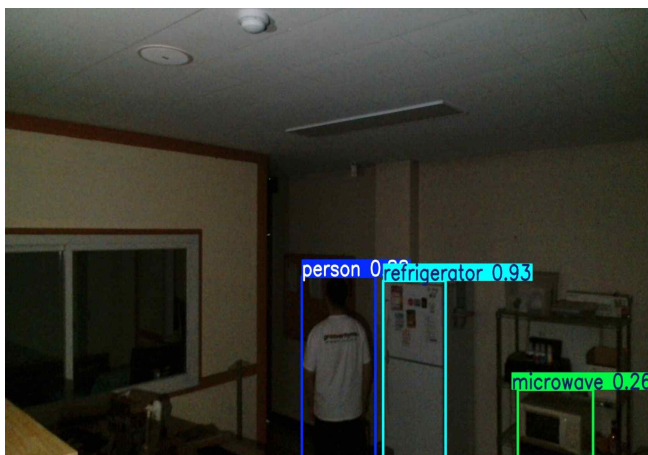
YOLOv8n은 어두운 이미지에서 person 0.90, refrigerator 0.81 수준의 confidence를 보였고, YOLOv8s는 person 0.90, person 0.74수준으로 나타났다.

이는 조도가 낮은 환경에서도 사람 탐지는 가능하지만, 밝은 환경보다 탐지 신뢰도가 낮아질 수 있음을 보여준다.

yolov8n



yolov8s



5) 오탐지 결과 분석

두 모델 모두 사람 탐지는 안정적으로 수행하였으나, 주변 사물에 대한 오탐지도 확인되었다.

특히 파티션이나 가구 일부가 refrigerator로 탐지되는 경우가 있었으며, YOLOv8n에서는 refrigerator 0.75~0.88, YOLOv8s에서는 refrigerator 0.83~0.93 수준으로 나타났다. 또한 일부 책상 영역은 bed 0.53~0.77로 탐지되었다.

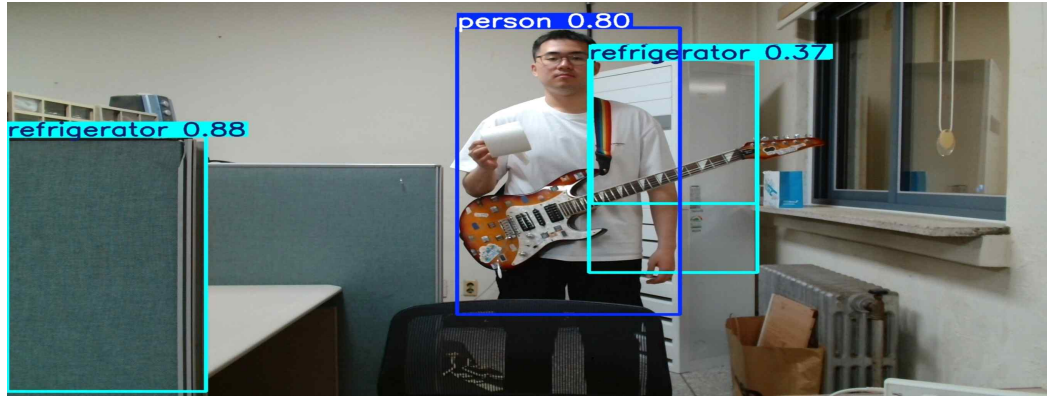
--> YOLO 모델이 객체의 실제 용도를 판단하는 것이 아니라 학습된 이미지 패턴과 유사도를 기반으로 객체를 분류하기 때문에 발생한 것으로 볼 수 있다.

이러한 오탐지는 본 프로젝트의 침입 감지 목적에는 불필요한 결과이므로, 이후 classes=[0] 옵션을 적용하여 person 클래스만 탐지하도록 수정하였다.

yolov8s 책장을 침대로 탐지



yolov8n 파티션이나 에어컨을 냉장고로 감지



7. YOLOv8n과 YOLOv8s 비교 종합

항목	YOLOv8n	YOLOv8s
모델 크기	경량 (nano)	중형 (small)
처리 속도	0.12 ~ 0.19초	0.21 ~ 0.35초
Confidence 범위	0.80 ~ 0.95	0.85 ~ 0.96
오탐지 경향	유사 수준	유사 수준
추천 상황	실시간 처리 우선	탐지 정확도 우선

수치만 놓고 보면 YOLOv8s가 사람 탐지에서 약간 더 높은 confidence를 보이는 경우가 많았다.

YOLOv8n은 사람을 대체로 0.80~0.95 범위에서 탐지하였고, YOLOv8s는 0.85~0.96 범위에서 탐지하였다.

특히 근거리 정면 이미지에서는 YOLOv8s가 person 0.96까지 나타나 YOLOv8n보다 조금 더 높은 신뢰도를 보였다.

처리 속도는 주요 비교 이미지 기준 YOLOv8n은 약 0.12~0.19초, YOLOv8s는 약 0.21~0.35초로 나타났다. YOLOv8n은 경량 모델이라 속도 면에서 유리하다.

따라서 실시간 영상 감시처럼 빠른 처리가 중요한 경우에는 YOLOv8n이 적합하고, 처리 속도보다 탐지 안정성이 더 중요할 경우에는 YOLOv8s가 적합하다.

본 프로젝트의 실내 침입 감지 목적에서는, Raspberry Pi의 처리 환경과 실시간성을 고려할 때 YOLOv8n이 우선 적용 모델로 적합하며, 추후 서버 성능 여유가 있을 경우 YOLOv8s로 전환을 검토할 수 있다.

8. 사람만 탐지하도록 코드 수정

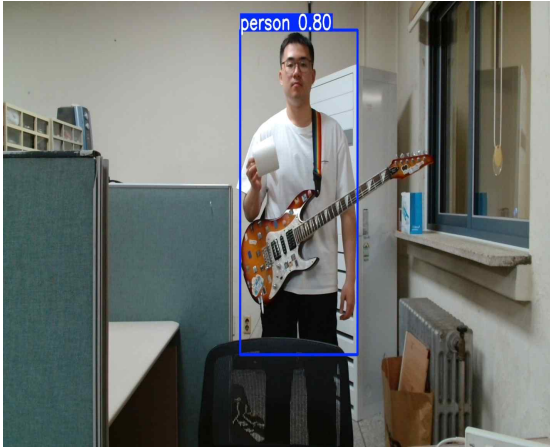
초기 실험에서는 YOLO 모델의 기본 탐지 성능과 오탐지 양상을 확인하기 위해 전체 객체 탐지를 수행하였다.

이후 본 프로젝트의 핵심 목적이 실내 침입 감지를 위한 사람 탐지임을 고려하여, person 클래스만 탐지하도록 코드를 수정하였다.

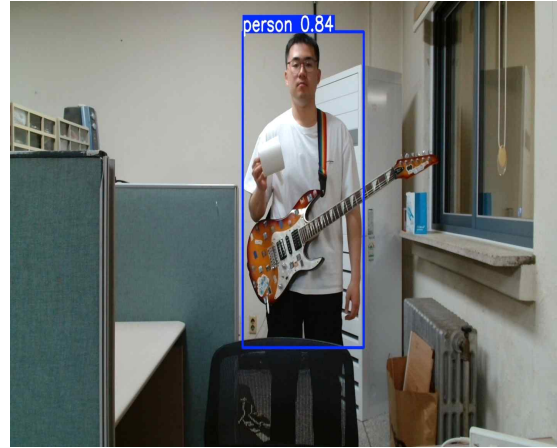
YOLO 탐지 실행 코드에 `classes=[0]` 옵션을 추가하여 사람 객체만 탐지하도록 수정하였다.

```
project= results 는 결과를 results 폴더에 저장
name=model_name은 모델별로 결과 폴더를 나누기 위한 설정
"""
for img_path, category in image_files:
    start_time =time.time()
    # YOLO 객체 탐지 실행
    results =model(
        str(img_path),
        classes=[0], # classes=[0]은 YOLOv8에서 person 클래스만 탐지하라
        save=True,
        project="results",
        name=f"{model_name}_person_only", # 예전 이미지와 섞일 수 있으니 저장 폴더 이름을 바꿈.
        exist_ok=True
    )
    """
    end_time = time.time()은 탐지 종료 시간을 기록
    processing_time은 탐지에 걸린 시간을 초 단위로 계산한 값
    """
```

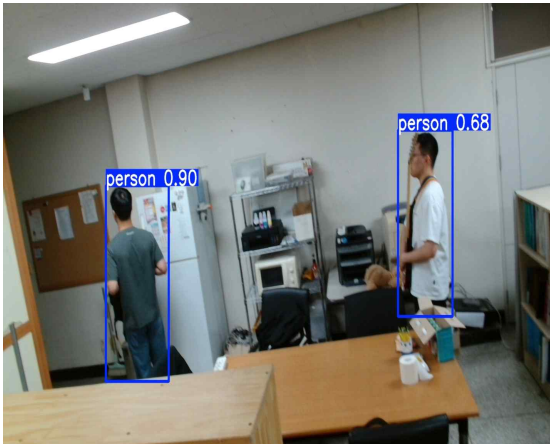
yolov8n



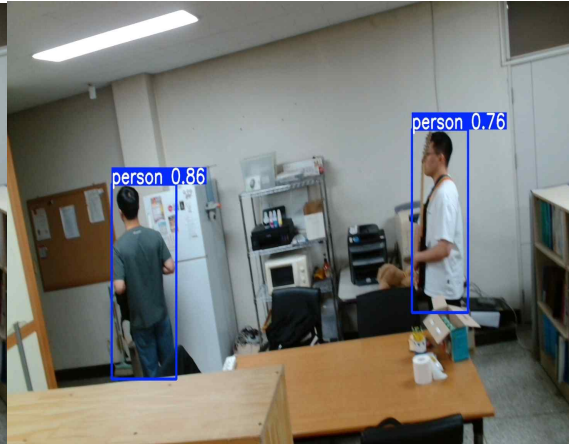
yolov8s



yolov8n



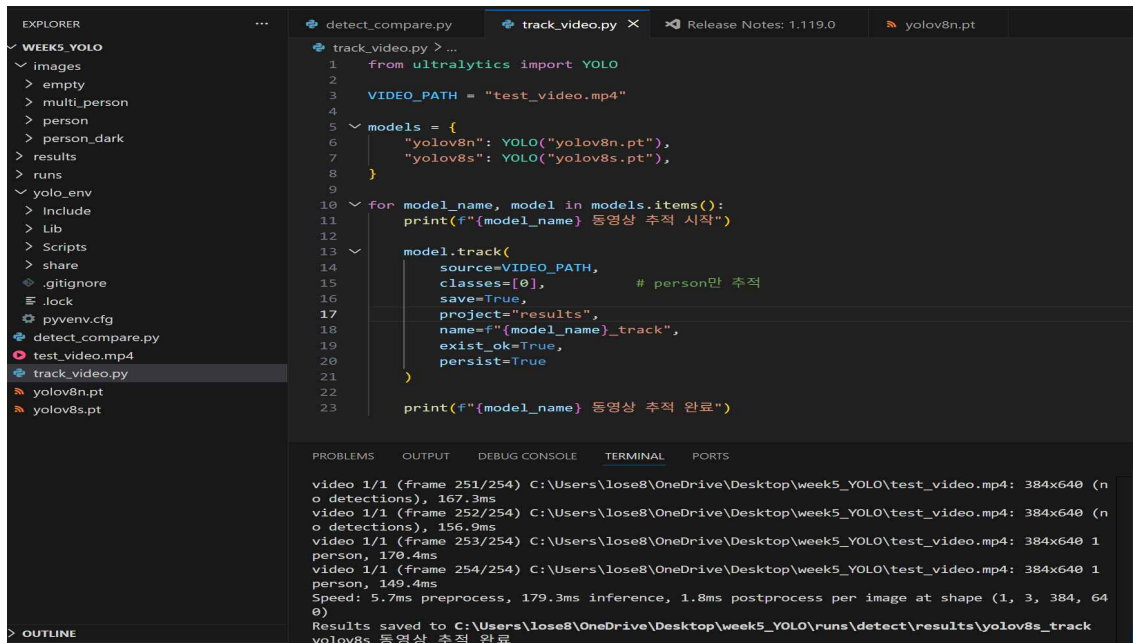
yolov8s



이를 통해 불필요한 객체 탐지 결과를 줄이고, 사람의 존재 여부를 중심으로 침입 여부를 판단할 수 있습니다.

9. 추가 확인: 동영상 및 실시간 웹캠 탐지

추가적으로 YOLOv8의 동영상 객체 추적 기능을 확인하기 위해 짧은 동영상 파일을 입력으로 사용하여 person 클래스 추적 가능성을 검토하였습니다.



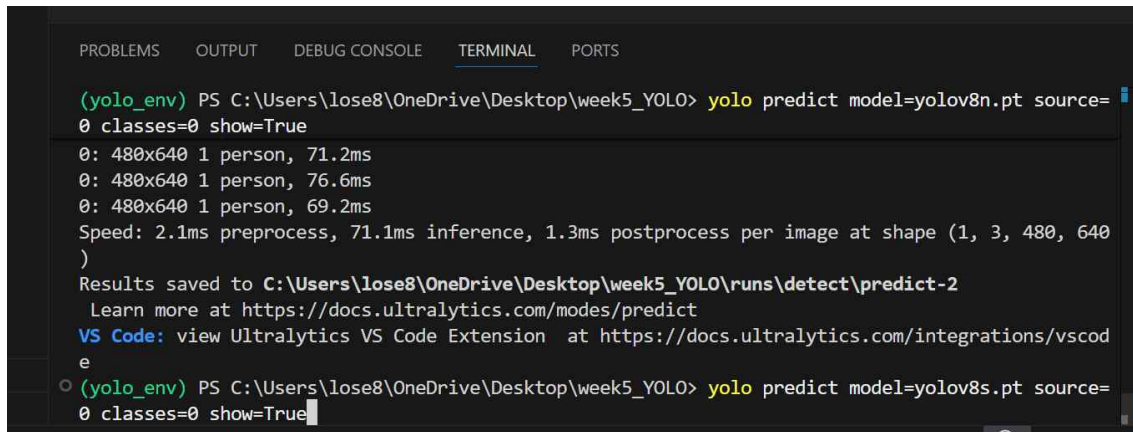
```
1 from ultralytics import YOLO
2
3 VIDEO_PATH = "test_video.mp4"
4
5 models = {
6     "yolov8n": YOLO("yolov8n.pt"),
7     "yolov8s": YOLO("yolov8s.pt"),
8 }
9
10 for model_name, model in models.items():
11     print(f"{model_name} 동영상 추적 시작")
12
13     model.track(
14         source=VIDEO_PATH,
15         classes=[0], # person만 추적
16         save=True,
17         project="results",
18         name=f"{model_name}_track",
19         exist_ok=True,
20         persist=True
21     )
22
23     print(f"{model_name} 동영상 추적 완료")
```

video 1/1 (frame 251/254) C:\Users\lose8\OneDrive\Desktop\week5_YOLO\test_video.mp4: 384x640 (no detections), 167.3ms
video 1/1 (frame 252/254) C:\Users\lose8\OneDrive\Desktop\week5_YOLO\test_video.mp4: 384x640 (no detections), 156.9ms
video 1/1 (frame 253/254) C:\Users\lose8\OneDrive\Desktop\week5_YOLO\test_video.mp4: 384x640 1 person, 170.4ms
video 1/1 (frame 254/254) C:\Users\lose8\OneDrive\Desktop\week5_YOLO\test_video.mp4: 384x640 1 person, 149.4ms
Speed: 5.7ms preprocess, 179.3ms inference, 1.8ms postprocess per image at shape (1, 3, 384, 640)
Results saved to C:\Users\lose8\OneDrive\Desktop\week5_YOLO\runs\detect\results\yolov8s_track
yolov8s 동영상 추적 완료

동일한 동영상에 대해 YOLOv8n과 YOLOv8s 모델을 각각 적용하여 사람 객체 탐지 결과를 비교하였다. 두 영상은 모두 1920×1080 해상도, 30fps, 약 8.47초 길이로 동일한 조건에서 생성되었다.

실험 결과 두 모델 모두 영상 내 사람 객체를 탐지할 수 있었으며, 본 프로젝트의 실내 침입 감지 목적에는 두 모델 모두 적용 가능함을 확인하였다.

노트북에 연결된 웹캠 화면을 실시간으로 받아서 YOLO가 사람을 탐지하는 방식



```
(yolo_env) PS C:\Users\lose8\OneDrive\Desktop\week5_YOLO> yolo predict model=yolov8n.pt source=0 classes=0 show=True
0: 480x640 1 person, 71.2ms
0: 480x640 1 person, 76.6ms
0: 480x640 1 person, 69.2ms
Speed: 2.1ms preprocess, 71.1ms inference, 1.3ms postprocess per image at shape (1, 3, 480, 640)
Results saved to C:\Users\lose8\OneDrive\Desktop\week5_YOLO\runs\detect\predict-2
Learn more at https://docs.ultralytics.com/modes/predict
VS Code: view Ultralytics VS Code Extension at https://docs.ultralytics.com/integrations/vscod
e
(yolo_env) PS C:\Users\lose8\OneDrive\Desktop\week5_YOLO> yolo predict model=yolov8s.pt source=0 classes=0 show=True
```

실시간 웹캠 탐지 실행 명령어는 다음과 같다.

```
yolo predict model=yolov8n.pt source=0 classes=0
show=True
```

여기서 model은 사용할 YOLO 모델을 의미하며, source=0은 PC에 연결된 기본 웹캠을 입력 장치로 사용한다는 의미이다. classes=0은 person 클래스만 탐지하도록 제한하는 옵션이며, show=True는 탐지 결과 화면을 실시간 창으로 출력하는 옵션이다.

동일한 웹캠 환경에서 비교했을 때, YOLOv8s 모델이 YOLOv8n 모델보다 사람 객체에 대한 confidence 값이 더 높게 나타나는 경향을 보였다.

YOLOv8s는 YOLOv8n보다 모델 크기가 크고 특징 추출 능력이 높기 때문에, 실시간 웹캠 화면에서도 사람 객체를 더 안정적으로 탐지하는 모습을 보였다.

다만 YOLOv8s는 YOLOv8n보다 연산량이 많기 때문에 처리 시간이 더 길어질 수 있다. 따라서 본 프로젝트처럼 실내 침입 감지를 목적으로 할 경우, 정확도를 우선하면 YOLOv8s가 적합하고, 속도와 가벼운 실행 환경을 우선하면 YOLOv8n이 적합하다고 판단된다.

출처: [동영상과 실시간 객체 탐지 방법 외 여러가지 방법들 제시](#)

10. 결론 및 6주차 계획

결론

5주차 실험 결과, PC 서버 환경에서 YOLOv8n과 YOLOv8s 모델이 모두 정상적으로 실행되었으며, Raspberry Pi에서 촬영·전송한 이미지에 대해 사람 객체를 탐지할 수 있음을 확인하였다.

YOLOv8n은 처리 시간이 짧아 실시간 처리에 유리하였고, YOLOv8s는 일부 이미지에서 더 높은 confidence를 보여 탐지 안정성 측면에서 장점이 있었다.

또한 classes=[0] 옵션을 적용하여 사람 객체만 탐지하도록 수정함으로써, 실내 침입 감지 시스템의 목적에 맞는 탐지 구조로 개선하였다.

따라서 본 프로젝트에서는 실시간성과 경량 실행 가능성을 우선 고려하여 YOLOv8n을 우선 적용 모델로 선정하고, 향후 PC 서버 성능이 충분하거나 탐지 안정성이 더 요구되는 경우 YOLOv8s 적용을 검토할 예정이다.

6주차(5.12) 계획

등록 인물의 얼굴 데이터를 수집하고, OpenCV 기반 얼굴 검출 및 특징 추출 방식을 설계한다.

등록 인물과 미등록 외부인을 구분하기 위한 얼굴 인식 구조를 설계하며,

탐지 결과를 저장하기 위한 DB 구조도 함께 설계할 예정이다. 이를 통해 단순 사람 탐지를 넘어, 허가된 인물과 외부 침입자를 자동으로 구분하는 기능을 구현하는 것을 목표로 한다.